

Zartags Writings

Ausarbeitung zum Web-Entwicklungsprojekt

Ronny Wittmer (Matrikelnummer: 313387)

Melissa Armbruster (Matrikelnummer: 313275)

Abgabedatum: 25. Juli 2026

Inhaltsverzeichnis

1	Einleitung	4
1.1	Projektidee	4
1.2	Motivation	4
1.3	Zielgruppe	4
2	Technologie-Stack	5
2.1	Frontend	5
2.2	Backend	5
2.3	Datenhaltung	6
2.4	Entwicklungs- und Testwerkzeuge	7
3	Architektur	8
3.1	Komponentenstruktur	8
3.2	API-Design	9
3.3	Datenbankschema	10
3.4	Diagramme	11
4	Umsetzung pro Meilenstein	14
4.1	Meilenstein 1	14
4.2	Meilenstein 2	14
4.3	Meilenstein 3	14
4.4	Zuordnung von VL-Konzepten zum Code	14
5	Testing & Qualitätssicherung	15
5.1	Teststrategie	15
5.2	Was wird getestet	15
5.3	Exemplarische Ergebnisse	16
6	Betrieb	17
6.1	Start der Anwendung	17
6.2	Konfiguration	17
6.3	Betriebsmodus	18
7	Reflexion & Fazit	19
7.1	Was lief gut	19
7.2	Was würden wir anders machen	19
7.3	Was wurde gelernt	19
A	Anhang: Screenshots der fertigen App	20
B	Formales und Quellenhinweise	21
B.1	Formale Anforderungen	21

B.2 Quellen	21
-----------------------	----

Abbildungsverzeichnis

1	Komponentendiagramm der Anwendung	11
2	Sequenzdiagramm: Login und geschützter Datenabruf	11
3	Vereinfachtes ER-Diagramm des Domänenmodells	12
4	Autorisierungs- und Request-Flow für schreibende Endpunkte	12
5	Platzhalter: Übersichtsseite der Anwendung	20

Tabellenverzeichnis

1	Zentrale API-Endpunkte	9
2	Relevante Umgebungsvariablen im Entwicklungsbetrieb	18

Codeverzeichnis

1	Backend-Testlauf	16
2	Frontend-Testlauf	16
3	Ausgewählte gezielte Testläufe	16
4	Backend lokal starten	17
5	Frontend lokal starten (zweites Terminal)	17

1 Einleitung

Zielumfang: ca. 1–2 Seiten.

1.1 Projektidee

Kurzbeschreibung von *Zartags Writings* als Tool zur strukturierten Verwaltung von Kampagneninhalten.

1.2 Motivation

Warum das Projekt entstanden ist und welches Problem im Spielalltag gelöst werden soll.

1.3 Zielgruppe

Für wen die Anwendung entwickelt wurde (z. B. DMs, Spielerinnen und Spieler).

2 Technologie-Stack

Der Technologie-Stack deckt die Anforderungen des Projekts (*Single-Page-Webanwendung mit Login, Rollen/Rechten und persistenter Datenhaltung*) mit überschaubarem Betriebsaufwand abdeckt. Im Fokus stehen dabei: schnelle Entwicklung, gute Testbarkeit und eine klare Trennung zwischen Benutzeroberfläche, API-Logik und Persistenz.

2.1 Frontend

Das Frontend basiert auf **React** mit **TypeScript** und wird über **Vite** gebaut und lokal ausgeführt.

React React passt hier, weil die Anwendung stark zustandsgetrieben ist: Anmeldestatus, Rollen im Campaign-Kontext, Entity-Listen und Bearbeitungszustände müssen konsistent über mehrere Seiten hinweg verwaltet werden. Durch die komponentenbasierte Struktur konnten wiederverwendbare UI-Bausteine (*Layout, Header, Campaign-Komponenten*) klar getrennt und wartbar umgesetzt werden.

TypeScript im Frontend TypeScript reduziert Integrationsfehler zwischen API und UI, insbesondere bei verschachtelten Nutzerdaten und Campaign-Strukturen. Das Projekt nutzt explizite Typen für zentrale Datenobjekte (z. B. *AuthUser, Campaign-API-Typen*), was die Weiterentwicklung erleichtert und Refactorings sicherer macht. Gerade bei rollenabhängiger Darstellung (*DM/EDITOR/PLAYER*) verbessert das die Verständlichkeit der Logik deutlich.

React Router Für die Navigation wird **react-router-dom** eingesetzt. Als Single-Page-Ansatz vermeidet dies vollständige Seiten-Reloads und erlaubt eine saubere Trennung zwischen öffentlichen Seiten (*Login/Registrierung*) und geschützten Bereichen (*Campaign-Ansichten, Verwaltung*). Dadurch bleibt die Nutzerführung schnell und konsistent.

Vite als Build- und Dev-Server Vite passt als Build-Tool, da es in der Entwicklung sehr kurze Start- und Aktualisierungszeiten liefert. Für dieses Projekt ist zusätzlich der konfigurierte `/api`-Proxy entscheidend: Frontend-Anfragen werden im Dev-Betrieb transparent an das Backend weitergeleitet. Das vereinfacht die lokale Entwicklung und vermeidet CORS-Probleme durch abweichende Ports.

2.2 Backend

Das Backend wurde mit **Node.js**, **Express** und **TypeScript** umgesetzt. Es stellt die REST-API bereit, kapselt die Geschäftslogik (*Authentifizierung, Rollenprüfung, Campaign-Operationen*) und steuert den Zugriff auf die Datenbank.

Node.js Node.js passt gut zum Frontend-Stack, da Sprache und Ökosystem über beide Schichten hinweg konsistent sind. Das reduziert Reibungsverluste im Team, vereinfacht Tooling und ermöglicht eine schnelle Iteration in den Meilensteinen.

Express Express ist als bewusst schlankes Framework hier passend. Für die Projektanforderungen — klar definierte REST-Endpunkte, Cookie-basierte Authentifizierung, Middleware für CORS/JSON/Parser — liefert Express genug Struktur, ohne unnötige Komplexität einzuführen. Die Routen sind nach Domänen getrennt (`authRoutes`, `userRoutes`, `campaignRoutes`, `entityRoutes`, `memberRoutes`), was die Wartbarkeit erhöht.

Auth- und Security-Bausteine Für Authentifizierung und Sicherheit werden mehrere spezialisierte Bibliotheken kombiniert:

- **jsonwebtoken:** Signieren und Verifizieren von JWTs,
- **cookie-parser:** Zugriff auf den Token im `HttpOnly-Cookie`,
- **bcrypt:** sicheres Hashing von Passwörtern,
- **cors:** kontrollierte Freigabe erlaubter Origins.

Die Entscheidung für JWT im `HttpOnly-Cookie` reduziert das Risiko, Tokens im Frontend-Code oder Local Storage offenzulegen.

2.3 Datenhaltung

Die Persistenz basiert auf **SQLite** in Kombination mit **Prisma** als ORM.

SQLite SQLite passt, weil die Anwendung im Projektkontext lokal, kompakt und ohne externen Datenbankserver betrieben werden soll. Für den vorgesehenen Umfang (Einzelinstantz, moderate Datenmengen, Entwicklungs-/Demobetrieb) bietet SQLite ein gutes Verhältnis aus Einfachheit und Zuverlässigkeit. Die Einstiegshürde ist gering, da Setup und Deployment ohne separate DB-Infra auskommen.

Prisma Prisma übernimmt die typsichere Datenzugriffsschicht und bildet das Domänenmodell explizit ab (`User`, `Campaign`, `CampaignMember`, `Entity`, `ContentBlock`, ...). Die wichtigsten Vorteile in diesem Projekt:

- **Klare Modellierung:** Relationen und Constraints sind direkt im Schema dokumentiert.
- **Typsichere Queries:** weniger Laufzeitfehler beim Zugriff aus dem TypeScript-Backend.
- **Migrationsunterstützung:** Schema-Änderungen bleiben versioniert und reproduzierbar.

Migrationskonzept Die Datenbankstruktur wird über Prisma-Migrationen verwaltet (`prisma/migrations/...`). Beim Start in Docker wird die Migration automatisiert ausgeführt (`prisma migrate deploy`), wodurch Entwicklungs- und Laufzeitumgebung konsistent bleiben. Dieses Vorgehen minimiert manuelle Setup-Fehler und erhöht die Reproduzierbarkeit.

2.4 Entwicklungs- und Testwerkzeuge

Neben den Kerntechnologien ergänzen Tools die Codequalität, Entwicklungsfluss und Nachvollziehbarkeit unterstützen.

Tests mit Vitest Sowohl Frontend als auch Backend nutzen **Vitest**. Im Backend werden API-Routen in einer Node-Umgebung getestet, im Frontend Komponenten- und Interaktionslogik in einer `jsdom`-Umgebung. Ergänzend kommen **Testing Library** und **Supertest** zum Einsatz, um UI-Verhalten bzw. HTTP-Endpunkte realitätsnah zu prüfen. Die Entscheidung für denselben Test-Runner in beiden Teilprojekten reduziert Komplexität im Tooling.

Codequalität: ESLint + Prettier Im Frontend sorgen **ESLint** (inkl. React- und TypeScript-Regeln) und **Prettier** für konsistente Codequalität und einheitliche Formatierung. Über **simple-git-hooks** wird vor Commits automatisch linting/formatting ausgeführt. So werden Stil- und Basisqualitätsprobleme früh abgefangen.

TypeScript-Tooling und Dev-Workflow Das Backend nutzt **tsx watch** für schnellen TypeScript-Dev-Betrieb ohne separaten Build-Schritt. Im Frontend übernimmt Vite denselben Zweck. Durch diesen direkten Feedback-Zyklus konnten Features pro Meilenstein inkrementell umgesetzt und getestet werden.

Containerisierung mit Docker Compose Für einen reproduzierbaren Start des Gesamtsystems wird **Docker Compose** genutzt. Frontend und Backend laufen als getrennte Services mit klaren Umgebungsvariablen (`DATABASE_URL`, `JWT_SECRET`, `CORS_ORIGIN`, `VITE_API_PROXY_TARGET`). Dadurch ist das Projekt unabhängig vom lokalen Host-Setup schnell lauffähig und für Abgabe/Demo stabil reproduzierbar.

3 Architektur

3.1 Komponentenstruktur

Die Anwendung folgt einer klassischen **3-Schichten-Struktur: Frontend (Präsentation), Backend (API + Fachlogik) und Persistenz (SQLite über Prisma)**. Die Schichten sind klar getrennt und kommunizieren ausschließlich über HTTP-Requests auf `/api/*`.

Frontend-Schicht Das Frontend ist als Single-Page-Anwendung mit React umgesetzt. Die Routen in `frontend/src/App.tsx` bilden die fachlichen Bereiche ab: Authentifizierung (`/login`, `/register`), Campaign-Übersicht (`/campaigns/:slug`) sowie Entity-Ansichten und Bearbeitung.

Eine zentrale Rolle übernimmt der `JWTAuthContext`: Er hält den globalen Authentifizierungszustand (`user`, `isLoggedIn`, `loading`, `error`), führt Login/Logout aus und lädt den aktuellen Benutzer über `/api/user`. Dadurch bleibt Auth-Logik aus den Seitenkomponenten ausgelagert und wiederverwendbar.

Für API-Aufrufe wird ein gemeinsamer Fetch-Wrapper (`frontend/src/lib/api.ts`) genutzt. Dieser setzt standardmäßig `credentials: include`, damit das JWT-Cookie bei geschützten Requests mitgesendet wird.

Backend-Schicht Das Backend ist modular in Routenpakete aufgeteilt, die in `backend/src/server/app.ts` registriert werden: `authRoutes`, `userRoutes`, `campaignRoutes`, `entityRoutes`, `memberRoutes`, `healthRoutes`.

Die Route-Module kapseln jeweils eine fachliche Domäne:

- **Auth:** Registrierung, Login, Logout,
- **User:** aktuelles Profil lesen/löschen,
- **Campaigns:** anlegen, beitreten, laden, Join-Code regenerieren,
- **Entities:** Templates, Listen, Details und CRUD je Entity-Typ,
- **Members:** Rollen- und Anzeigenamenverwaltung innerhalb einer Campaign.

In `backend/src/server/entities.ts` liegt zentrale Domain-Logik, die von mehreren Routen wiederverwendet wird, z. B.: `loadCampaignForUser`, `serializeEntity`, `canEditCampaign`, `canManageCampaign`. So wird verhindert, dass Autorisierungs- und Transformationslogik in mehreren Dateien dupliziert wird.

Persistenz-Schicht Der Datenzugriff erfolgt ausschließlich über Prisma. Die Prisma-Client-Instanz wird zentral in `config.ts` erzeugt und über den Better-SQLite3-Adapter mit

DATABASE_URL verbunden. Das reduziert Kopplung, da Routen keine eigene DB-Verbindung aufbauen.

3.2 API-Design

Die API ist als **REST-orientierte JSON-API** unter /api umgesetzt. Für geschützte Endpunkte wird die Middleware `authenticateToken` verwendet, die den JWT aus dem `HttpOnly-Cookie` validiert und den User-Kontext an den Request hängt.

Konventionen

- Ressourcenorientierte Pfade, z. B. `/api/campaigns/:slug`.
- HTTP-Methoden gemäß Operation (GET, POST, PUT, PATCH, DELETE).
- Fehlerantworten mit passendem Statuscode und JSON-Fehlermeldung (`{ error: ... }`).
- Rollenprüfung serverseitig vor schreibenden Aktionen.

Methode	Pfad	Schutz	Zweck
POST	<code>/api/register</code>	nein	Benutzer registrieren
POST	<code>/api/login</code>	nein	JWT-Cookie setzen
POST	<code>/api/logout</code>	nein	JWT-Cookie löschen
GET	<code>/api/user</code>	ja	aktuellen Benutzer laden
GET	<code>/api/campaigns</code>	ja	Campaign-Liste des Users
POST	<code>/api/campaigns</code>	ja	neue Campaign erstellen
POST	<code>/api/campaigns/join</code>	ja	per Join-Code beitreten
GET	<code>/api/campaigns/:slug</code>	ja	Campaign inkl. Entities laden
POST	<code>/api/campaigns/:slug/regenerate-join-code</code>	ja (DM/Owner)	Join-Code erneuern
GET/POST/PUT/DELETE	<code>/api/campaigns/:slug/entities/...</code>	ja (rollenabhängig)	Entity-CRUD
PATCH	<code>/api/campaigns/:slug/members/:memberUserId</code>	ja (DM/Owner)	Mitgliedsdaten/Rolle ändern

Tabelle 1: Zentrale API-Endpunkte

Zentrale Endpunkte

Autorisierungslogik Die Autorisierung ist zweistufig aufgebaut:

1. **Authentifizierung** über JWT-Cookie (`authenticateToken`).
2. **Fachliche Berechtigungen** über Rollenprüfungen: `canEditCampaign` erlaubt Owner, DM und Editor; `canManageCampaign` erlaubt Owner und DM.

Damit werden unzulässige Änderungen serverseitig abgefangen, auch wenn ein Client manipulierte Requests sendet.

3.3 Datenbankschema

Das Datenmodell ist auf zwei Kernanforderungen ausgelegt: **(1) Mitglieder- und Rollenverwaltung pro Campaign** und **(2) flexible Inhaltsstruktur für unterschiedliche Entity-Typen**.

Kernentitäten

- **User**: registrierte Benutzer mit Login-Daten.
- **Campaign**: Kampagne mit Owner, Metadaten und Join-Code.
- **CampaignMember**: Zuordnung User ↔ Campaign inkl. Rolle.
- **Entity**: fachliches Objekt (PC, NPC, MAGIC_ITEM, LOCATION).
- **EntityField**: strukturierte Header-Felder einer Entity.
- **ContentBlock**: Inhaltskarten pro Entity (Text, Liste, Attribute, Bild).
- **ContentListItem** und **ContentAttribute**: Unterelemente für Listen und Attribute.

Relationen und Constraints

- Ein User kann in vielen Campaigns Mitglied sein; eine Campaign hat viele Mitglieder (`CampaignMember` als Join-Tabelle).
- Eine Campaign hat viele Entities; eine Entity gehört genau zu einer Campaign.
- Eindeutigkeit wird über fachliche Schlüssel gesichert, u. a. `Campaign.slug`, `Campaign.joinCode`, `User.email` sowie (`campaignId`, `type`, `slug`) für Entities.
- Cascading-Deletes entfernen abhängige Daten konsistent (z. B. Entity → Fields/Blocks).

Das Schema kombiniert damit klare Normalisierung für Beziehungen (`User`, `Campaign`, `CampaignMember`) mit einer flexiblen Content-Struktur für heterogene Kampagneninhalte.

3.4 Diagramme

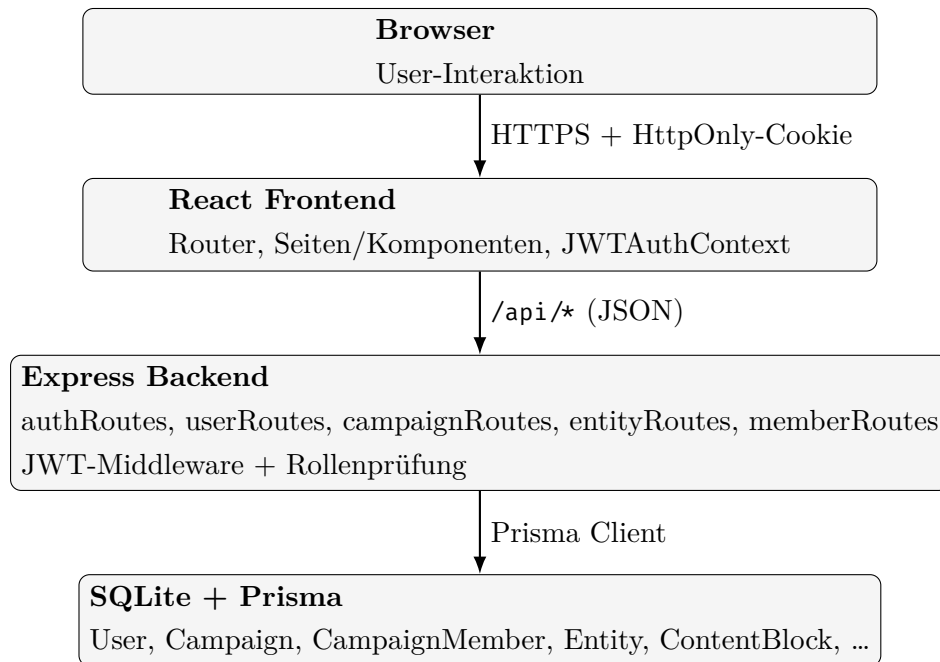


Abbildung 1: Komponentendiagramm der Anwendung

Das Komponentendiagramm zeigt die klare Entkopplung der Schichten. Das Frontend kennt die Datenbank nicht direkt, sondern ausschließlich die HTTP-Schnittstelle. Geschäftsregeln und Rechteprüfung bleiben vollständig im Backend.

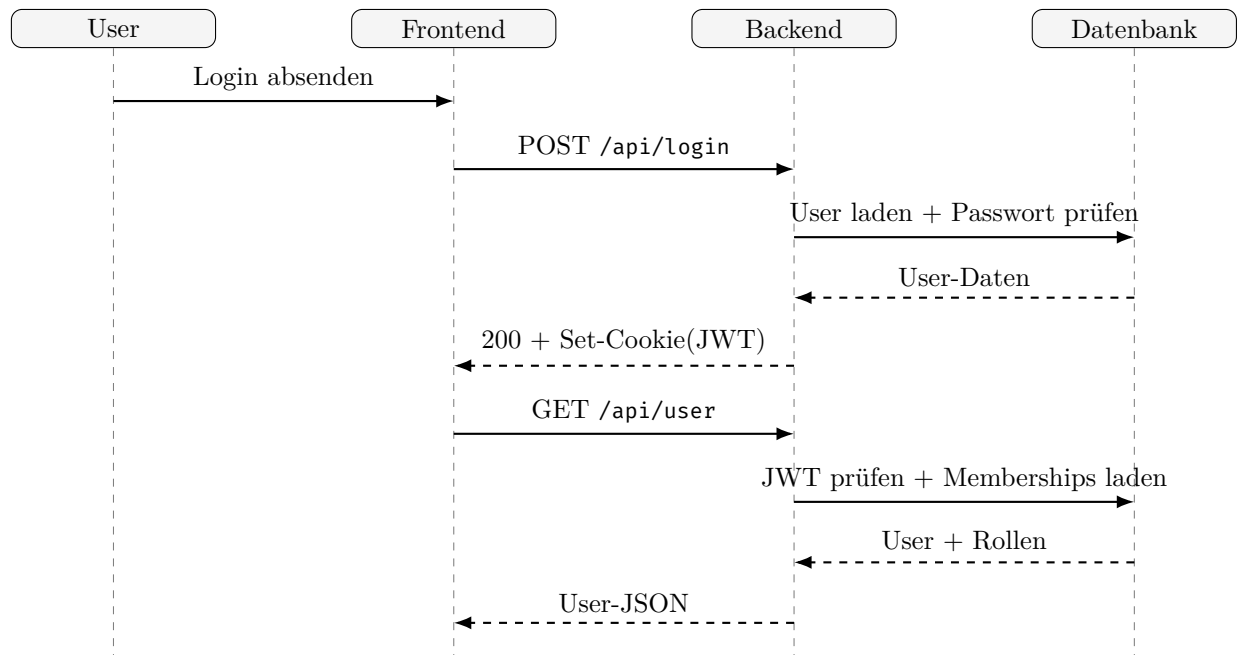


Abbildung 2: Sequenzdiagramm: Login und geschützter Datenabruf

Der Ablauf verdeutlicht, dass der Token nicht im JavaScript-Speicher abgelegt wird, sondern als HttpOnly-Cookie übertragen wird. Folgezugriffe auf geschützte Ressourcen nutzen denselben Mechanismus und werden zusätzlich durch rollenbasierte Prüfungen abgesichert.

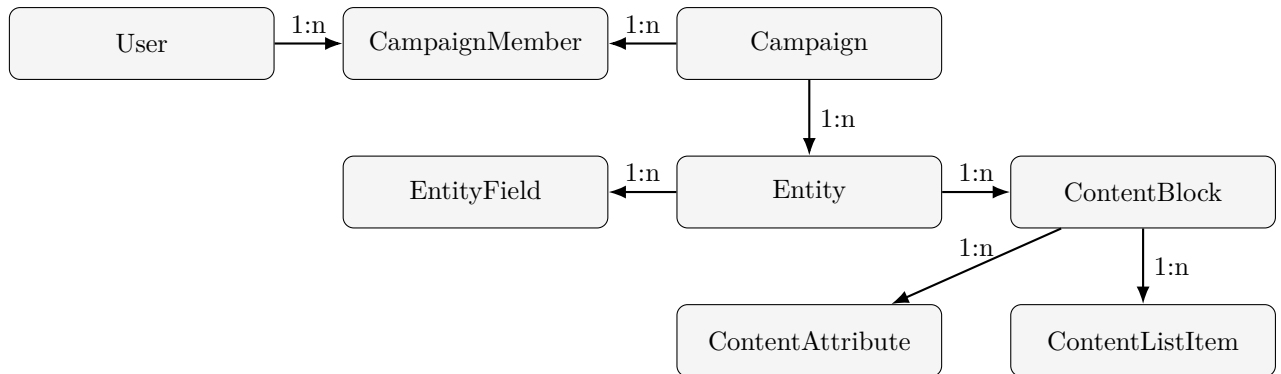


Abbildung 3: Vereinfachtes ER-Diagramm des Domänenmodells

Das ER-Diagramm zeigt die zwei zentralen Stränge der Persistenz: Mitgliedschaft mit Rollen (User–CampaignMember–Campaign) und inhaltliche Modellierung der Campaign-Daten (Campaign–Entity–ContentBlock).

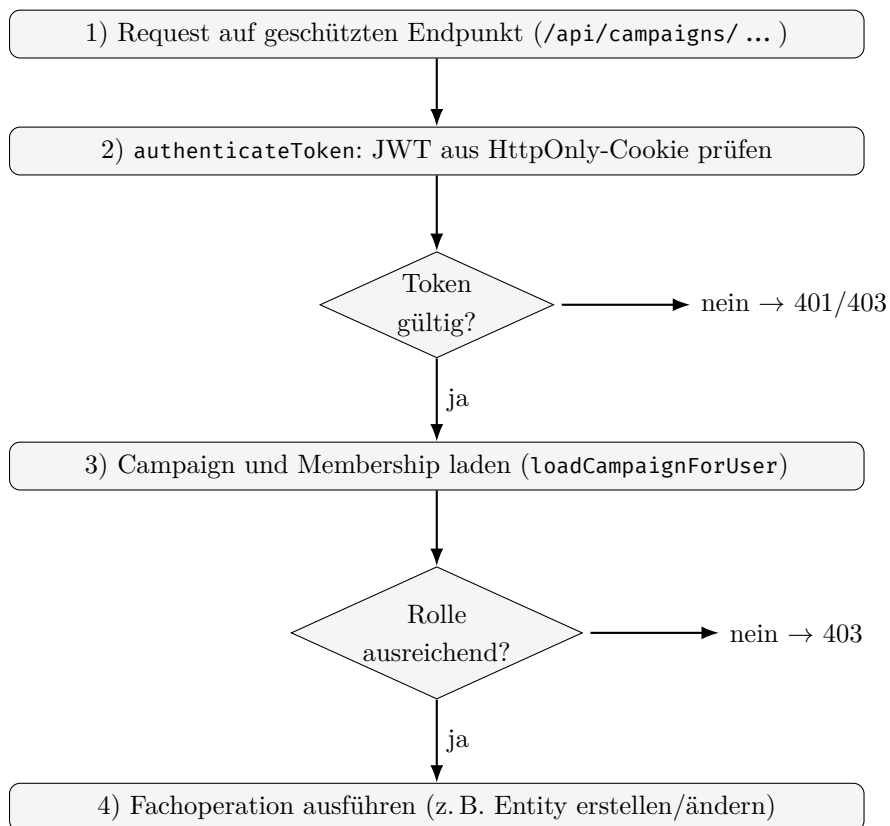


Abbildung 4: Autorisierungs- und Request-Flow für schreibende Endpunkte

Der Ablauf macht sichtbar, dass Autorisierung strikt serverseitig erfolgt: ohne gültigen Token oder ausreichende Rolle wird die Aktion vor der Fachoperation abgelehnt.

4 Umsetzung pro Meilenstein

Zielumfang: ca. 6–8 Seiten.

4.1 Meilenstein 1

Was umgesetzt wurde, welche Anforderungen abgedeckt wurden und welche Vorlesungskonzepte dabei verwendet wurden.

4.2 Meilenstein 2

Fortschritt, technische Entscheidungen und Verweise auf relevante Codebereiche.

4.3 Meilenstein 3

Erweiterungen, Verbesserungen und Lessons Learned innerhalb der Umsetzung.

4.4 Zuordnung von VL-Konzepten zum Code

Konkrete Zuordnung: Konzept → Datei/Modul/Feature.

5 Testing & Qualitätssicherung

5.1 Teststrategie

Die Qualitätssicherung kombiniert drei Ebenen: **Unit-nahe Logiktests**, **API-Integrationstests** und **Frontend-Komponenten-/Flow-Tests**. Ziel ist nicht nur die Prüfung einzelner Funktionen, sondern vor allem die Absicherung der kritischen Nutzerpfade: Registrierung, Login, Rollen-/Rechteprüfung und Campaign-Operationen.

Backend-Strategie Im Backend werden Routen mit `supertest` gegen die Express-App getestet. Dadurch lassen sich HTTP-Statuscodes, Response-Bodies, Cookie-Handling und Autorisierungsregeln realitätsnah prüfen. Die Prisma-Zugriffe werden in den Tests gezielt gemockt, sodass Fehlerfälle (z. B. Unique-Constraint-Verletzungen) reproduzierbar testbar sind.

Frontend-Strategie Im Frontend werden mit Vitest + Testing Library Seiten und Context-Logik getestet. Fokus liegt auf sichtbar nutzerrelevantem Verhalten: Formularvalidierung, API-Aufrufe, Navigation sowie Fehler- und Erfolgsfälle. Externe Abhängigkeiten (`fetch`, Routing) werden kontrolliert gemockt.

Warum diese Aufteilung Die Kombination aus API-Integrationstests und Frontend-Interaktionstests liefert ein gutes Verhältnis aus Aussagekraft und Wartbarkeit: Routenfehler, fehlende Berechtigungsprüfungen und fehlerhafte Client-Flows werden früh erkannt, ohne den Aufwand eines vollständigen End-to-End-Teststacks.

5.2 Was wird getestet

Backend-Abdeckung Die Backend-Tests decken zentrale Sicherheits- und Geschäftsregeln ab:

- **Authentifizierung:** Login erfolgreich/fehlgeschlagen, JWT-Cookie wird gesetzt, Logout liefert erwartete Antwort.
- **Autorisierung:** geschützte Endpunkte liefern `401` ohne Token und `403` bei ungültigem Token.
- **Rollenrechte in Campaigns:** Editor darf Join-Code nicht regenerieren und keine Member-Rollen ändern; DM darf beide Operationen ausführen.
- **Registrierung:** Passwortregeln und Duplikatbehandlung (`409` bei bereits registrierter E-Mail).
- **Join-Code-Logik:** Format des Join-Codes sowie Retry-Verhalten bei Unique-Konflikten.

Frontend-Abdeckung Die Frontend-Tests prüfen sowohl Seitenlogik als auch API-Helfer:

- **Auth-Context:** `login()` und `logout()` rufen die richtigen API-Endpunkte mit korrekten Request-Optionen auf.
- **Register-Flow:** schwache Passwörter werden clientseitig blockiert, starke Passwörter führen zum Register-Request.
- **Home-Flow:** Campaigns laden, Join per Code und Weiterleitung auf die Overview-Seite.
- **API-Wrapper (`apiFetch`):** `credentials: include`, Header-Handling und Fehler-Mapping (JSON-error bzw. Fallback-Text).

5.3 Exemplarische Ergebnisse

Die folgenden Auszüge stammen aus den aktuellen Testläufen:

```
cd backend
npm test

Test Files  4 passed (4)
Tests       21 passed (21)
```

Listing 1: Backend-Testlauf

```
cd frontend
npm test

Test Files  5 passed (5)
Tests       11 passed (11)
```

Listing 2: Frontend-Testlauf

```
# Backend (Autorisierungs- und Logout-Fälle)
npm --prefix backend test -- src/server.routes.test.ts
# Ergebnis: 11 passed

# Frontend (API-Wrapper-Tests)
npm --prefix frontend test -- src/__tests__/api.test.ts
# Ergebnis: 3 passed
```

Listing 3: Ausgewählte gezielte Testläufe

Interpretation Die Ergebnisse bestätigen, dass die zentralen Sicherheits- und Geschäftsregeln stabil umgesetzt sind: Auth-Cookie-Flow funktioniert erwartungsgemäß, unautorisierte Zugriffe werden korrekt abgefangen und rollenbasierte Schreibrechte serverseitig durchgesetzt. Auf Frontend-Seite zeigen die Tests, dass API-Aufrufe konsistent aufgebaut werden und Fehler dem UI in auswertbarer Form zur Verfügung stehen.

6 Betrieb

6.1 Start der Anwendung

Für den Entwicklungsbetrieb ist **Docker Compose** der primäre Startweg, da Frontend, Backend und Persistenzpfad in einem reproduzierbaren Setup zusammengeführt werden.

Betrieb mit Docker Compose Der Start erfolgt mit den folgenden Befehlen:

```
cp .env.example .env
docker compose up
```

Nach dem Start stehen die zentralen Endpunkte bereit:

- Frontend: <http://localhost:5173>
- Backend: <http://localhost:3000>
- Health-Endpoint: <http://localhost:3000/api/health>

Im Compose-Betrieb führt der Backend-Service vor dem Start automatisch `prisma migrate deploy` aus. Damit ist sichergestellt, dass das Datenbankschema dem aktuellen Migrationsstand entspricht.

Lokaler Betrieb ohne Docker Alternativ kann die Anwendung lokal in zwei Prozessen betrieben werden:

```
cd backend
npm ci
npx prisma migrate deploy
npm run dev
```

Listing 4: Backend lokal starten

```
cd frontend
npm ci
npm run dev
```

Listing 5: Frontend lokal starten (zweites Terminal)

Im lokalen Betrieb ist ein `backend/.env` mit den notwendigen Variablen erforderlich (siehe section 6.2).

6.2 Konfiguration

Die Anwendung wird über Umgebungsvariablen konfiguriert. Für den Compose-Betrieb dient `.env.example` als Vorlage.

Variable	Beispielwert	Bedeutung
JWT_SECRET	dev-secret-change-me	Secret für Signierung und Verifikation der JWTs im Backend.
CORS_ORIGIN	http://localhost:5173	Erlaubte Origin(s) für CORS-Requests auf die API.
DATABASE_URL	file:./data/dev.db	SQLite-Datenbankpfad des Backends.
VITE_API_PROXY_TARGET	http://backend:3000	Ziel für den /api-Proxy des Vite-Dev-Servers.

Tabelle 2: Relevante Umgebungsvariablen im Entwicklungsbetrieb

Voraussetzungen Für den Betrieb werden Node.js und npm benötigt. Im Docker-Modus sind zusätzlich Docker Engine und Docker Compose erforderlich. Die Persistenz erfolgt auf SQLite-Basis und benötigt keinen separaten Datenbankserver.

6.3 Betriebsmodus

Die Anwendung ist als **Entwicklungs- und Demo-Betrieb** ausgelegt. Frontend und Backend laufen als getrennte Prozesse/Container, kommunizieren über HTTP und nutzen ein Cookie-basiertes Session-Modell mit JWT.

Typischer Workflow

1. Services starten (Compose oder lokal).
2. Benutzer über /register anlegen und anmelden.
3. Campaign erstellen oder per Join-Code beitreten.
4. Inhalte pflegen (abhängig von der Rolle DM/EDITOR/PLAYER).

Betriebsrelevante Eigenschaften

- Daten werden persistent in SQLite gespeichert.
- Schemaänderungen werden über Prisma-Migrationen ausgerollt.
- Geschützte Endpunkte erfordern gültige JWT-Cookies.
- Die Trennung von Frontend und Backend erlaubt klare Fehlersuche (UI-/API-/DB-Ebene).

7 Reflexion & Fazit

Zielumfang: ca. 2 Seiten.

7.1 Was lief gut

Positive Aspekte in Planung, Teamarbeit und technischer Umsetzung.

7.2 Was würden wir anders machen

Verbesserungspotenziale für Architektur, Vorgehen oder Priorisierung.

7.3 Was wurde gelernt

Wichtigste technische und organisatorische Erkenntnisse aus dem Projekt.

A Anhang: Screenshots der fertigen App

Zielumfang: ca. 2–3 Seiten.

Hier werden Screenshots der finalen Anwendung mit kurzen Bildunterschriften eingebunden.

Abbildung 5: Platzhalter: Übersichtsseite der Anwendung

B Formales und Quellenhinweise

B.1 Formale Anforderungen

- Ausgabe als PDF mit sauberem Layout.
- Deckblatt mit Namen, Matrikelnummern, Projekttitel und Abgabedatum.
- Quellenangaben bei maßgeblich genutzten externen Ressourcen.

B.2 Quellen

Alle externen Quellen werden im Literaturverzeichnis und bei Bedarf direkt im Fließtext referenziert.